- *I don't want to build my own identity system.* I'm glad to hear it. Using an external identity provider like Azure Active Directory or Auth0 is a great way of shifting the responsibility and risk associated with storing users' personal information to a third party.

Any time you're considering adding user management to your ASP.NET Core application, I'd recommend looking at Identity as a great option for doing so. In the next section I'll demonstrate what Identity provides by creating a new Razor Pages application using the default Identity UI. In section 23.4 we'll take that template and apply it to an existing app instead, and in sections 23.5 and 23.6 you'll see how to override the default pages.

23.3 Creating a project that uses ASP.NET Core Identity

I've covered authentication and Identity in general terms, but the best way to get a feel for it is to see some working code. In this section we're going to look at the default code generated by the ASP.NET Core templates with Identity, how the project works, and where Identity fits in.

23.3.1 Creating the project from a template

You'll start by using the Visual Studio templates to generate a simple Razor Pages application that uses Identity for storing individual user accounts in a database.

TIP You can create a similar project using the .NET CLI by running `dotnet new webapp -au Individual`. The Visual Studio template uses a LocalDB database, but the `dotnet new` template uses SQLite by default. To use LocalDB instead, run `dotnet new webapp -au Individual --use-local-db`.